

SIH26054 – DRDO (Robotics & Drones)

AI-Enabled Real-Time Digital Twin System for MALE UAV Aero Piston Engines

Organization: DRDO

Theme: Robotics and Drones

Problem Statement ID: SIH26054

MALE UAVs (Medium Altitude Long Endurance) rely on piston engines for long-duration ISR, communication relay, maritime surveillance, and strategic missions. Propulsion reliability is critical; a mid-flight engine failure can cause mission loss or UAV loss. Current maintenance is often reactive or schedule-based, not fully leveraging real-time data for predictive maintenance.

Expected solution: an AI-enabled real-time digital twin that continuously monitors engine health (vibration, temperature, pressure, RPM, fuel flow, etc.), mirrors physical engine behavior, detects anomalies, predicts faults, estimates remaining useful life (RUL), and improves mission reliability via early warnings, maintenance suggestions, and operator dashboards.

Team Structure (6 members)

Each member owns a clear, demo-worthy module that maps directly to the problem statement.

#	Role	Primary focus	Key deliverables
M1	Digital Twin Core (Physics Model)	Virtual engine model	RPM/CHT/EGT/oil P&T/fuel flow/vibration model; healthy vs faulty behavior plots
M2	Residual Engine & Prognostics Core	Residuals, health index, baseline RUL	Residual engine, health index (0–100), rule-based RUL, features for ML
M3	Data Pipeline, Telemetry & Fault Injection	Data streaming, schemas, fault scenarios	Streaming harness, mock CAN schema, fault injector, replay CSVs
M4	AI/ML: Anomaly Detection, Classification & Explainability	ML models for detection/diagnosis	Anomaly detector, fault classifier, explainability overlay, metrics
M5	Dashboard, Mission Reliability & Operator UX	UI and mission-reliability scoring	Streamlit dashboard, health gauge, alert panel, mission-reliability score, reports
M6	System Architecture, Integration, Runtime & Deployment	End-to-end system and deployment	Architecture diagram, simulation orchestrator, integration, Docker, deployment, reset, testing

Detailed Role Descriptions

M1 — Digital Twin Core (Physics Model)

Goal: Build the virtual engine that mirrors real behavior.

Tasks

- Design state variables and equations for RPM, CHT, EGT, oil pressure/temp, fuel flow, vibration.
- Implement time-stepping simulation in Python (throttle → RPM → temperatures/pressures with realistic dynamics).
- Add basic mission-profile effects (altitude, ambient temperature).
- Output clean time series for M2 and M3 to consume.

Presentation

- Explain the twin concept.
- Show plots: healthy vs faulty engine behavior from the model.

M2 — Residual Engine & Prognostics Core

Goal: Turn twin outputs and sensor data into smart, interpretable prognostic signals.

Tasks

- Build the residual engine:
- Compare predicted (M1) vs actual (simulated sensor) values.
- Generate residual signals as primary anomaly features.
 - Design and implement:
- Health index (0–100) from residual magnitude and patterns.
- Basic RUL mapping from health decay (before M3's ML version).
 - Work with M4 to:
- Turn residuals + health index into ML features.
- Add uncertainty to RUL (ensemble or simple confidence bands).

Presentation

- Show residual plots and how they highlight faults earlier than raw thresholds.
- Explain health index and baseline RUL logic before ML refinements.

M3 — Data Pipeline, Telemetry & Fault Injection

Goal: Own all data flow and fault scenario generation.

Tasks

- Build a streaming harness (fixed-rate publisher emitting sensor-like time series).
- Define a mock CAN/telemetry schema (message IDs, fields, units, sampling rate).
- Implement a fault injection module (misfire, injector fault, overheating, oil-pressure drop, vibration spike).
- Add environmental scenarios (hot day, high altitude, rapid throttle transients).
- Generate and save pre-recorded fault CSVs for replay.

Presentation

- Show the data schema and live stream.
- Demonstrate fault injection and replay of a recorded mission.

M4 — AI/ML: Anomaly Detection, Classification & Explainability

Goal: Build ML models for detection, diagnosis, and explanations.

Tasks

- Define the feature set (raw sensors + residuals from M2 + engineered features).
- Implement anomaly detection (Isolation Forest / autoencoder).
- Implement fault classifier (misfire vs overheating vs oil drop, etc.).
- Add explainability (feature importance or simple rule-based explanations).
- Evaluate with metrics (precision/recall on injected faults).

Presentation

- Live demo: healthy → fault → detection → classification.
- Show an explainability panel: which sensors/features drove each alert.

M5 — Dashboard, Mission Reliability & Operator UX

Goal: Build the operator/maintenance interface and mission-reliability scoring.

Tasks

- Build a Streamlit dashboard (real-time plots, health gauge, alert panel).
- Implement mission-reliability scoring (health + mission profile → success probability).
- Create operator view (live health, alerts, mission risk) and maintenance view (diagnosis, RUL, recommendations).
- Generate a mission report after each simulated flight.

Presentation

- Walk through the dashboard during a fault scenario.
- Explain mission-reliability scoring with an example mission.

M6 — System Architecture, Integration, Runtime & Deployment

Goal: Own the overall system design, integration glue, simulation runtime, and deployment.

Tasks

- Design and document system architecture (blocks, data flow, interfaces).
- Implement simulation runtime / orchestrator (runs M1's model, uses M3's streaming/fault injection, exposes API for M4/M5).
- Set up repo structure, shared config, CI skeleton, containerization (Docker + docker-compose).
- Implement persistent logging (runs, faults, health, RUL) and reset API/button.
- Own integration testing (define test scenarios, verify end-to-end behavior).
- Deploy to a public hosting platform; prepare live URL and backup local demo.
- Create architecture diagram and system narrative for slides.

Presentation

- Present architecture diagram and explain module integration.
- Show simulation runtime, integration testing approach, and live deployed system.

7-Day Hackathon Sprint Plan

Day	M1 (Twin Core)	M2 (Residual + Prognostics)	M3 (Data + Faults)	M4 (AI/ML)	M5 (Dashboard + Mission)	M6 (System + Deploy)
1	Define state variables; implement basic RPM/CHT/EGT model	Define residual computation; start health index formula	Define telemetry schema; build basic streaming harness	Define feature schema with M2/M3	Dashboard skeleton wired to raw stream	Repo + architecture draft; Docker base; orchestrator skeleton
2	Add oil P/T, fuel flow, vibration; add mission-profile effects	Implement residual engine; start health index code	Implement basic fault injection (overheat, oil drop)	Baseline anomaly detector on healthy data	Live parameter plots; fault-toggle controls	Containerize services; first staging deploy
3	Tune model dynamics; prepare healthy/faulty plots	Finalize health index; start rule-based RUL	Add environmental scenarios; save first replay CSVs	Add fault classifier; start explainability logic	Health gauge + alert panel	CI tests/lint; staging URL live
4	Buffer / refine model	Tune health/RUL; prepare residual + health plots	Finalize fault injector; prepare demo scenarios	Integrate residuals + health as features	Mission-reliability scoring logic	Persistent logging; reset API
5	Buffer / docs	Buffer / docs	Buffer / docs	Uncertainty-quantified RUL with M2	Operator + maintenance views; mission report draft	Integration pass; production deploy candidate
6	Buffer / bugfix	Buffer / bugfix	Buffer / bugfix	Buffer / bugfix	Full HMI pass; architecture diagram support	End-to-end testing; load-test demo path
7	Buffer / demo prep	Buffer / demo prep	Buffer / demo prep	Buffer / demo prep	Rehearse demo on deployed URL	Final deploy freeze; backup local demo; system deck

USP / Differentiators to Highlight

- **Physics-informed residual twin** (M1+M2): anomaly detection based on predicted-vs-actual divergence, not just thresholds.
- **Interpretable health index & baseline RUL** (M2): single 0–100 score and simple RUL extrapolation.
- **Explainable AI diagnostics** (M4): alerts accompanied by “why” (which sensors/features).
- **Mission-reliability scoring** (M5): combines health with mission profile to estimate success probability.
- **Fully deployed, integrated system** (M6): live URL, reset, replay, offline fallback, architecture diagram.

Notes for Presentation Flow

Suggested order (6 × ~1–2 minutes):

1. M1: Digital twin model and engine behavior plots.
2. M2: Residual engine, health index, baseline RUL logic.
3. M3: Data pipeline, fault injection, replay demo.
4. M4: AI detection, classification, explainability demo.
5. M5: Dashboard walk-through + mission-reliability scoring.
6. M6: System architecture, integration, deployment, and live link.

This ensures each member has substantial, visible work that maps directly to the DRDO problem statement.